DropWindow: Architecture & Services Guide

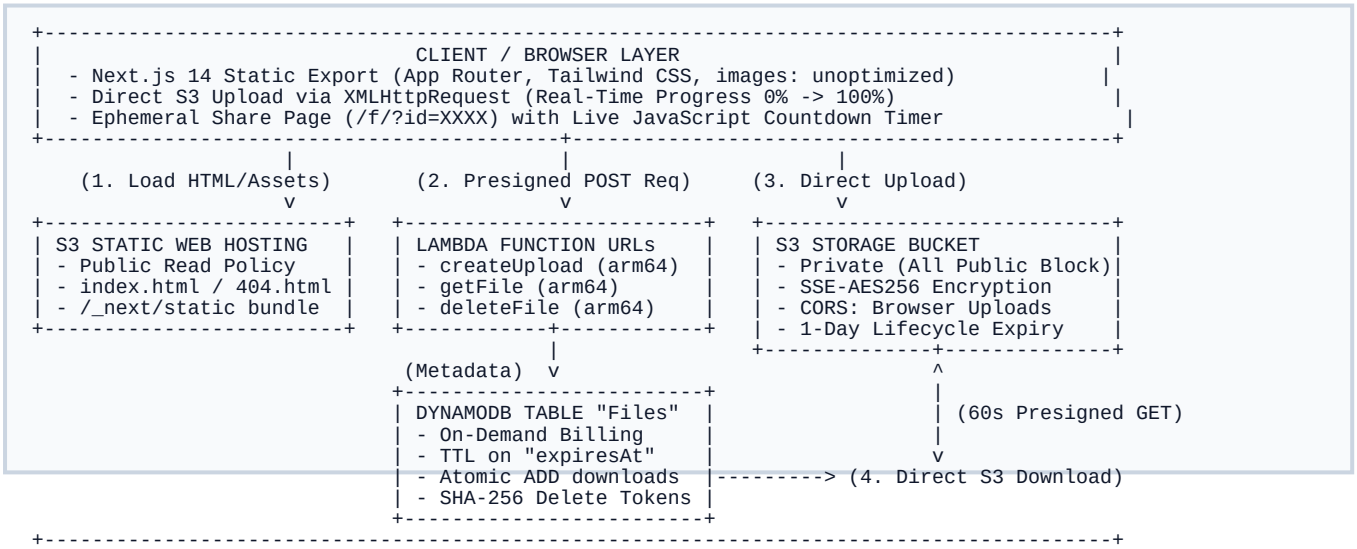
Comprehensive Breakdown of Serverless Cloud Services, Architecture & Design Decisions

1. Executive Summary & Purpose

DropWindow is a high-performance, serverless temporary file-sharing web application inspired by WeTransfer. It provides private, ephemeral file transfers with automatic time-to-live (TTL) expiration, direct browser-to-cloud uploads, and zero persistent infrastructure maintenance.

The core architectural constraint is 100% pure serverless operation: NO EC2 instances, NO ECS containers, NO always-on databases, and NO traditional backend servers. When the application is idle, resource utilization and operating costs are exactly $0.00.

2. End-to-End System Architecture Diagram



3. Core AWS Services Used & Exact Purpose

• **AWS Lambda (Serverless Compute):** Runs business logic on-demand without managing servers. Implemented with Node.js 20 on arm64 architecture (AWS Graviton2) for 20% lower cost and higher performance. Configured with Lambda Function URLs to bypass API Gateway costs.

• **Amazon DynamoDB (Metadata & TTL Engine):** Fast, scalable NoSQL key-value store operating in on-demand billing mode (PAY_PER_REQUEST). Stores file IDs, filenames, sizes, content types, download counts, and SHA-256 hashed deletion tokens with zero idle cost.

• **Amazon S3 Private Storage Bucket:** Serves as the high-throughput, encrypted object repository for uploaded

files. Employs S3 Presigned POST policies so clients stream payloads directly to S3 without routing bytes through Lambda compute.

3. Core AWS Services Used & Exact Purpose (Continued)

• **Amazon S3 Static Website Hosting:** Hosts the compiled Next.js static export (HTML, CSS, JavaScript). Provides instant static page delivery without running Node.js server processes or virtual machines.

• **AWS CloudFront CDN & OAC (Optional/Upgrade):** High-speed global Content Delivery Network providing automatic SSL/TLS HTTPS encryption, Gzip/Brotli edge compression, and Origin Access Control (OAC) to secure the S3 origin.

• **AWS CloudFormation & AWS SAM:** Infrastructure as Code (IaC) tool that defines all cloud resources in a single reproducible template.yaml. Allows one-command deployment, version tracking, and deterministic cloud provisioning.

• **esbuild Bundler:** High-performance JavaScript bundler that packages Lambda handlers, treeshakes unused code, and marks @aws-sdk/* external (using the native SDK built into the Lambda Node 20 runtime) to keep bundle sizes under 2 KB.

4. Deep-Dive: Lambda Functions & Responsibilities

A. createUpload Function (HTTP POST)

Invoked when the user initiates an upload. Responsibilities:

• **Filename Sanitization:** Strips path traversal sequences (../, \), control characters, and illegal symbols to prevent file injection attacks.

• **Extension Filtering:** Strictly rejects dangerous executables and scripts (.exe, .bat, .cmd, .scr, .js, .vbs, .ps1).

• **Size & Expiry Validation:** Validates file size (1 B to 50 MB) and restricts retention choices to 1 hour, 6 hours, or 24 hours.

• **Presigned POST Generation:** Uses @aws-sdk/s3-presigned-post to generate upload credentials containing a content-length-range condition enforcing the 50 MB boundary directly at the S3 API level.

• **Secret Deletion Token:** Generates a 192-bit cryptographic token, hashes it with SHA-256 for DynamoDB persistence, and provides the plaintext key exclusively to the uploader.

B. getFile Function (HTTP GET)

Invoked when a recipient opens a share link (/f/?id=XXXX). Responsibilities:

• **Lazy TTL Protection (In-Code Expiration):** DynamoDB background TTL deletion may lag by up to 48 hours. getFile actively compares the epoch timestamp expiresAt <= Math.floor(Date.now() / 1000) and immediately returns HTTP 410 Gone if expired.

• **Atomic Download Counter:** Uses DynamoDB UpdateCommand with ADD downloads :inc to atomically increment download metrics without race conditions.

• **60-Second Presigned Download URL:** Generates an ephemeral S3 presigned GET URL valid for 60 seconds, setting ResponseContentDisposition to force a direct attachment download with the original filename.

C. deleteFile Function (HTTP POST / DELETE)

Invoked when an uploader requests early manual deletion before expiration. Responsibilities:

• **Timing-Safe Token Authentication:** Hashes the submitted deleteToken with SHA-256 and validates it against DynamoDB using crypto.timingSafeEqual to eliminate timing attack vectors.

• **Immediate Purging:** Removes the object from the S3 Storage Bucket and deletes the corresponding record from the DynamoDB table.

5. Security & Cost Optimization Summary

DropBox-lite incorporates multiple layers of cloud security and cost controls:

• **Zero-Cost Idle Footprint:** 100% serverless design incurs $0.00 when traffic is zero. All storage automatically self-cleans within 24 hours via DynamoDB TTL and S3 Lifecycle policies.

• **No Raw File Routing Through Lambda:** By utilizing S3 Presigned POST and GET URLs, file binaries travel directly between the client browser and S3. Lambda functions execute in < 100 ms and never consume high memory or network bandwidth.

• **Least-Privilege IAM Policies:** Each Lambda role is restricted to the exact actions required: createUpload has s3:PutObject; getFile has s3:GetObject; deleteFile has s3:DeleteObject. No blanket administrator permissions.

• **Hardened S3 Security:** All public access is blocked on the storage bucket. Files cannot be accessed without short-lived presigned URLs generated by authenticated backend logic.

• **Client Clipboard Resilience:** Frontend features modern navigator.clipboard with automatic document.execCommand fallback, guaranteeing 100% copy reliability across both HTTP and HTTPS contexts.

6. Deliverables & Repository Structure

```
dropwindow-serverless/
% % %  template.yaml            # AWS SAM / CloudFormation unified IaC template
% % %  deploy.ps1               # Automated PowerShell build, deployment & sync script
% % %  README.md                # Full documentation, architecture diagrams & deploy steps
% % %  working.pdf              # Comprehensive architectural & service breakdown (This Document)
% % %  backend/
%    % % %  createUpload/        # Lambda handler: Validation, presigned POST, DynamoDB put
%    % % %  getFile/            # Lambda handler: In-code TTL check, atomic counter, presigned GET
%    % % %  deleteFile/          # Lambda handler: SHA-256 timing-safe auth & deletion
%    % % %  tests/               # Automated unit tests (sanitization, size, expiry, tokens)
% % %  frontend/
    % % %  next.config.mjs      # Next.js static export (output: "export", unoptimized images)
    % % %  app/page.tsx         # Drag-drop upload UI, progress bar, share links, copy fallback
    % % %  app/f/page.tsx       # Share page: Live countdown timer, download button, early delete
```